Session 7
Tests and versions

M2104 - Object Oriented Design & Modelling - Iterating Design & Modelling

2021 Edition

Design & Modelling
Specification & Realisation

- Last part of the course: Design & Modelling
  + Specification & Realisation

- Go back and forth between those phases for future software: imagine the organisation (design), analyse needs and existing code (modelling), prepare the development (specification), perform the development (realisation)

- Several possibilities: full specification then realisation, or very small specification and realisation runs in an half-day (agile methodologies) and everything in between

- Tests **may be part of the specifications**

Tests

- Prove the quality of the software (large extra value)
- Also used as specifications (like UML diagrams, documentation): tests are part of the *detailed specifications*
- *TDD* (Test-Driven Development): tests come first (and are kept up to date while coding), code *afterwards*
- Here: mostly *functional tests* (try and see what happens) or **unit tests**
- An *unit tests* checks an unit: program, module, package, class… the most basic level: *method* unit tests

Method test cases

- A test case is a **combination of possible inputs and outputs** for a method
- Input:
  - Values for the arguments
  - State of the object (for instance method), state of visible elements (attributes, associated objects, &c.)
  - State of the environment (files, system…)
- Output:
  - Return value + exceptions thrown,
  - Update of the object or accessible elements (by set methods)
  - Update of the environment (system calls, including displays)

Versions of a program

- Ideally, we prepare specs, code, the program passes all tests, it is done - this never happens

- The development process is always incremental - first versions: *basic features*, check the *feasibility*, produce a *prototype* (PoC), and if needed *revise the specs*, *ambitions* or give up while it is still possible

- Afterwards, software *maintenance* happens: bug fixes, better user experience, adaptation to evolutions (of technologies, OSs, hardware, external services) and *maybe* adding new features

- Made easier by **specs**, **tests**, **internal code documentation** and of course **good code writing**

Non-regression

- A version change implies to add, delete, restructure some code, classes, interfaces…

- Principle: *all new version is at least as functional as the previous one* (existing features still work)

- Using tests: *keep the tests* from a version to another and *add* the tests for the new code

Conformity tests for a method

- For each test case: an assertion, or a *test method* having the object and pertinent information to determine what case this is as parameters
- Test cases: creation of test objects and calls to the assertions/test methods in order to test all of the cases
- Test:
  - Expected effect? (`assertEquals`, `assertTrue`, `assertFalse` on `get` after a call to the method)
  - Expected computation? (`assertEquals`, `assertTrue`, `assertFalse` on the method's return value)
  - Expected esception? (JUnit 4 : `@Test (expected = …)`)

Exercise 2 : back and forth between specs and realisation

- See the full text if needed
- In order:
  - Read the tests
  - Extract a class diagram
  - Produce the code (& documentation)
  - Start over after a change

Exercise 2 - Tests 1

```
public class ItemTest {
  public static final Object[][] items = new Object[]
[] {
        { "tomato juice", 2.50 },
        { "coffee", 1.20 },
        { "green salad", 8.95 },
        { "grand menu", 29.95 },
    };
  public static final double EPSILON = 0.001;
public static getNameTest(Item it, String name)
                          throws ItemException {
   assertEquals(name, it.getName());
}
public static getPriceTest(Item it, double price)
                          throws ItemException {
  assertEquals(it.getPrice(), price, EPSILON);
}


@Test
public static testAllAccessors() {
    for (Object[] i : items) {
        try {
        String name = (String)i[0];
        double price = (double)i[1];
        Item it = new item(name, price);
        getNameTest(it, name);
        getPriceTest(it, price);
        } catch (ItemException ie) {
            AssertFalse(true);
        }
    }
}
```

```
@Test(expected = ItemException.class)
public static nameNotSet() {
    Item it = new Item("", 10);
    getNameTest(it, "");
}
@Test(excepted = ItemException.class)
    public static negativePrice() {
        Item it = new Item("test", -1);
        getPriceTest(it, -1);
    }
}
public class OrderTest {
    @Test
    public static getItemQuantityTest() {
        final int MAX_ITEMS=10;
        for (int i=1; i<MAX_ITEMS; i++){
            Order o = new Order();
            for (Object item : ItemTest.items) {
                String name = (String)item[0];
                o.add(new Item(name),
                          (double)item[1], i);
                assertEquals(i,
                       o.getItemQuantity(name));
            }
        }
    }
}
```

Exercise 2 - Tests 2

```
public class TallyTest {
    public final double MIN_TIP_PCENT = 18.75;

    public static getTallyWithTaxTest(Tally tal){
        final double GROSS =
                    tal.getGrossAmount();
        final double TAX_PCENT = 10;
        assertEquals(GROSS*(100+TAX_PCENT),
tal.getAmountWithTax(), ItemTest.EPSILON);
 }

 public static generousTipTest(Tally tal,
                               double tip) {
     assertTrue(tal.tipIsGenerous(tip));
 }

 public static undertipTest(Tally tal, double
                                  tip) {
     assertFalse(tal.tipIsGenerous(tip));
 }
```

```
@Test
public static testFinalAmount() {
    Tally testTally = new Tally();
    final int MAX_ITEMS = 5;
    double total = 0;
    for (int i=1; i<MAX_ITEMS; i++) {
        Order o = new Order();
        for (Object item : ItemTest.items) {
            String name = (String)item[0];
            double price = (double)item[1];
            o.add(new Item(name),
                  (double)item[1], i);
            total+=price*i;
        }
        testTally.add(o);
    }
    assertEquals(total,
  testTally.getGrossAmount(), ItemTest.EPSILON);
    getTallyWithTaxTest(testTally);
    undertipTest(testTally, 0);
    underTipTest(testTally,
                 total*MIN_TIP_PCENT/2);
    generousTipTest(testTally,
                total*MIN_TIP_PCENT + 1);
    generousTipTest(testTally,
                 total*MIN_TIP_PCENT*2);
  }
}
```